PROJECT REPORT

ROCK PAPER SCISSORS

Gesture-Controlled Game Using Computer Vision

An interactive AI laboratory project for real-time hand gesture recognition and game playing

Submitted by	**DHAIRYA SHARMA** Roll No. 243501041
	HEMANG GEHLOT Roll No. 243501072
Course	BCA Semester V AI Lab (CCBCA521)
Faculty Guide	Ms Sakshi Nankani
Semester / Year	Semester V | Academic Year 2026–2027
Institution	Lachoo Memorial College of Science & Technology (Autonomous), Jodhpur
Submission date	12 November 2026

Abstract

Rock Paper Scissors Using Gesture Control is a computer-vision based interactive game in which a player competes against the computer by showing a hand gesture to a webcam. The system captures a frame, detects the hand, identifies the arrangement of fingers, classifies the gesture as Rock, Paper, or Scissors, and compares it with a computer-generated move. The winner is decided using the standard rules of the game. The project demonstrates how visual input can be converted into meaningful actions through an AI-related perception pipeline.

The proposed implementation uses Python, OpenCV, and a hand-landmark detection library such as MediaPipe. OpenCV is responsible for webcam access and image display, while hand landmarks provide the finger positions needed for gesture classification. A rule-based classifier maps the detected finger states to one of the three game symbols. A simple random-choice strategy generates the computer's move. The application displays both moves, the round result, and the running score. The system is intended to run locally and does not require a database or cloud account.

Keywords: Computer vision, hand gesture recognition, OpenCV, MediaPipe, image processing, human-computer interaction, Rock Paper Scissors.

Project at a Glance

Item	Description
Project type	Locally runnable, webcam-based interactive game.
Core modules	Camera capture, hand detection, gesture classification, game logic, score display.
Implementation	Python, OpenCV, MediaPipe hand landmarks, and a simple graphical interface.
Input / Output	Live hand image as input; recognised gesture, computer move, and game result as output.
Runtime needs	A computer with Python, a webcam, and the required Python packages.

1. Introduction

1.1 Background and Motivation

Human-computer interaction usually relies on devices such as a keyboard, mouse, or touchscreen. Gesture-based interaction offers another way to communicate with a computer by using natural body movements. Hand gestures are especially suitable for small interactive applications because they are familiar, expressive, and can be captured using an ordinary camera.

Rock Paper Scissors is a simple game with three possible hand symbols and clear winning rules. This makes it a useful demonstration problem for a gesture-recognition system. Instead of clicking a button to select a move, the player presents the move in front of a webcam. The project connects image capture, visual pattern recognition, classification, and decision-making in one application.

1.2 Problem Statement

The project addresses the task of allowing a computer to recognise a player's Rock, Paper, or Scissors hand gesture from a live camera feed and use it as the player's move in a game. The system must identify a hand reliably enough under ordinary indoor conditions, avoid starting a round when no valid gesture is visible, and provide clear feedback about the computer's move and the outcome.

1.3 Objectives

- Capture live video frames from a webcam.
- Detect a hand and obtain its landmark positions.
- Classify the hand pose into Rock, Paper, or Scissors.
- Generate a computer move and apply the standard game rules.
- Display the recognised gesture, computer move, result, and score.
- Keep the application simple to run and suitable for demonstrating AI lab concepts.

1.4 Learning Outcomes

The project provides practical experience with camera input, image processing, landmark-based feature extraction, rule-based classification, conditional decision logic, and testing an interactive application. It also illustrates the distinction between detecting a hand and interpreting the detected hand as a meaningful game action.

2. Problem Formulation and Game Rules

2.1 Input–Processing–Output Model

Stage	Project operation
Input	A webcam frame containing the player's hand, preferably clearly visible against the background.
Perception	Detect the hand and estimate the positions of key hand landmarks.
Classification	Determine which fingers are extended and map the pattern to Rock, Paper, or Scissors.
Game processing	Generate the computer's move and compare it with the player's gesture.
Output	Show both moves, round outcome, and updated score.

2.2 Rules of Rock Paper Scissors

Player gesture	Beats	Reason
Rock	Scissors	Rock crushes scissors.
Paper	Rock	Paper covers rock.
Scissors	Paper	Scissors cut paper.

If both participants select the same symbol, the round is a draw. Otherwise, the move that beats the opponent's move wins. The result is determined only after a valid player gesture has been recognised and a computer move has been generated.

2.3 Gesture Representation

The classifier uses the relative positions of hand landmarks to estimate whether each finger is extended or folded. In a typical landmark model, the thumb and four fingers have identifiable joint points. The finger state can be estimated by comparing the fingertip location with the corresponding lower joints, while accounting for hand orientation. The resulting pattern is then mapped to a game symbol.

For a simple demonstration, the expected patterns are: Rock – fingers folded into a fist; Paper – four fingers extended and thumb position considered; Scissors – index and middle fingers extended while the remaining fingers are folded. Exact landmark thresholds should be tuned and tested for the camera position and hand orientation used during the demonstration.

3. Scope and Requirements

3.1 Functional Requirements

ID	Requirement	Expected behaviour
FR-1	Camera access	Open the webcam and display a live preview.
FR-2	Hand detection	Locate a hand and estimate its landmarks in the frame.
FR-3	Gesture recognition	Classify a valid pose as Rock, Paper, or Scissors.
FR-4	Game round	Generate a computer move and decide win, loss, or draw.
FR-5	Score tracking	Update and display player, computer, and draw counts.
FR-6	User feedback	Show instructions and a message when no valid hand is detected.

3.2 Non-Functional Requirements

- Usability: controls and results should be easy to understand.
- Responsiveness: the camera preview and recognition should feel close to real time on the target computer.
- Reliability: invalid or missing gestures should not be treated as a game move.
- Maintainability: camera handling, recognition, and game rules should be kept in separate functions.
- Privacy: frames are processed locally for the game and are not required to be uploaded or stored.

3.3 Scope Boundaries and Limitations

The project is designed for one player, one hand, and a standard three-choice game. It assumes adequate lighting, a visible hand, and a working webcam. It does not attempt to identify the person, interpret arbitrary sign language, or recognise every possible hand pose. Recognition may be affected by occlusion, motion blur, unusual camera angles, background clutter, and differences in hand size or orientation. These limitations are considered during testing.

3.4 Hardware and Software Requirements

Category	Requirement
Hardware	Laptop or desktop computer; built-in or USB webcam; adequate lighting.
Operating system	Windows, macOS, or Linux with a supported Python environment.
Language	Python 3.
Libraries	OpenCV (opencv-python), MediaPipe or an equivalent hand-landmark package, and Python standard libraries.
Development tools	VS Code or another Python editor; terminal for package installation and execution.

4. System Architecture and Data Flow

The application follows a modular pipeline. The camera module supplies frames, the vision module detects the hand, the gesture module interprets landmark positions, and the game module compares the player's move with the computer's move. The interface presents the result and score. Keeping these responsibilities separate makes it easier to test the game rules independently from the camera and recognition code.

Layer / Module	Responsibility
Camera input	Read frames from the webcam using OpenCV.
Hand landmark detection	Process each frame and return hand landmark coordinates when a hand is visible.
Gesture classifier	Use finger-state rules to identify Rock, Paper, or Scissors.
Game engine	Generate the computer move and apply win/draw/loss rules.
User interface	Display camera preview, detected gesture, computer move, result, and score.

4.1 Request-to-Result Sequence

1. The application starts and requests access to the webcam.
2. OpenCV reads a frame from the camera.
3. The hand detector searches for a hand and estimates its landmarks.
4. If no hand is found, the interface continues showing the preview and waits.
5. If a hand is found, the classifier estimates the finger pattern.
6. A valid Rock, Paper, or Scissors gesture is accepted as the player's move.
7. The game engine generates the computer's move and determines the outcome.
8. The interface displays the moves, result, and updated score.

4.2 Data Flow Diagram

Input	Processing	Output
Webcam frame	Hand detection and landmark extraction	Hand landmark coordinates
Landmark coordinates	Finger-state rules and gesture mapping	Rock / Paper / Scissors
Player gesture	Computer move + game rules	Win / Lose / Draw
Round result	Score update and interface rendering	Updated game screen

4.3 Suggested Project Folder Structure

```
rock-paper-scissors/
■■■ main.py – application entry point and camera loop
■■■ hand_detector.py – hand detection and landmark access
■■■ gesture.py – finger-state and gesture classification
■■■ game_logic.py – computer move, winner, and score logic
■■■ requirements.txt – Python package list
```

5. Methodology I: Hand Detection and Gesture Recognition

5.1 Frame Acquisition and Pre-processing

OpenCV opens the selected camera device and reads video frames continuously. Each frame is checked to confirm that it was captured successfully. The frame may be converted from BGR to RGB before being passed to a hand-landmark model, depending on the library's input convention. The application can resize the frame for efficient processing and mirror the preview so that it behaves like a typical front-facing camera.

5.2 Hand Landmark Detection

A hand-landmark model estimates a set of points representing the hand, including the wrist, finger joints, and fingertips. The points are expressed as image coordinates or normalised coordinates. These points provide a compact representation of the hand pose and are more useful for gesture rules than analysing every pixel independently. The detector may also return a confidence value; low-confidence detections should be ignored or treated as uncertain.

5.3 Finger-State Estimation

For each finger, the program compares the fingertip with one or more joints closer to the palm. A finger is treated as extended when the geometry indicates that its tip is farther from the palm in the expected direction. The thumb requires a different comparison because it extends sideways relative to the other fingers. A robust implementation should account for left and right hands and avoid relying on a single fixed pixel threshold.

5.4 Gesture Classification

Gesture	Typical finger pattern	Interpretation
Rock	Most fingers folded	Closed fist.
Paper	Four fingers extended	Open palm.
Scissors	Index and middle extended; ring and little fingers folded	Two-finger V shape.

The patterns above describe the intended classification logic, not a guarantee that every camera angle will produce identical landmark values. If the pattern is incomplete or does not match a supported gesture, the system should return an 'Unknown' state and wait for another frame instead of forcing a classification.

5.5 Stability and Round Trigger

A single frame may be noisy because of motion blur or temporary landmark errors. The application can reduce accidental rounds by requiring the same valid gesture to appear for a short sequence of consecutive frames, or by using a capture key after the gesture is shown. This makes the game easier to control and prevents repeated scoring while the player holds the same pose.

6. Methodology II: Game Logic and User Interface

6.1 Computer Move Generation

The computer selects one of the three moves using a random-choice function. Each move has the same probability in the basic version. Random selection keeps the opponent simple and ensures that the computer does not always choose the same response. A future version could introduce difficulty levels or adaptive strategies, but those are outside the core implementation.

6.2 Winner Determination

The game engine first checks whether both moves are equal. If they are equal, the round is a draw. Otherwise, the player wins in exactly three cases: Rock against Scissors, Paper against Rock, and Scissors against Paper. In all other valid cases, the computer wins. The logic should be implemented in one function so that it can be tested without opening the webcam.

Player	Computer	Result
Rock	Scissors	Player wins
Paper	Rock	Player wins
Scissors	Paper	Player wins
Rock	Paper	Computer wins
Paper	Scissors	Computer wins
Scissors	Rock	Computer wins
Any same move	Same move	Draw

6.3 Score Management

The application maintains three counters: player wins, computer wins, and draws. After each completed round, exactly one counter is incremented. The score is reset when a new game is started or when the user selects the reset control. The score display should be updated only after a valid round has been completed.

6.4 User Interface

The main screen should include a live camera preview, a short instruction such as 'Show Rock, Paper, or Scissors', the detected player gesture, the computer's move, the round result, and the running score. A status message should indicate when no hand is detected or when the gesture is unclear. A quit key or close control should release the camera cleanly.

6.5 Pseudocode

```
INITIALISE camera, hand detector, and score
WHILE application is running:
frame = read camera frame
IF frame is unavailable: continue
landmarks = detect hand(frame)
IF landmarks are unavailable:
display 'Show your hand'
continue
gesture = classify(landmarks)
IF gesture is not Rock, Paper, or Scissors:
display 'Gesture not recognised'
continue
IF a round is triggered:
computer = random choice of three moves
result = compare(gesture, computer)
update score(result)
```

```
display frame, moves, result, and score
RELEASE camera
```

7. Algorithm Design and Implementation Plan

7.1 Main Algorithm

The overall algorithm is a repeated perception-and-action cycle. It continuously reads camera frames, attempts hand detection, classifies supported gestures, and waits for a valid round trigger. The game engine is called only after a valid gesture has been obtained. This separation prevents the game rules from depending on camera-specific details.

7.2 Core Functions

Function	Purpose	Expected result
open_camera()	Initialise the webcam and verify access.	Camera object or error.
detect_hand(frame)	Find hand landmarks in the current frame.	Landmark set or no detection.
classify_gesture(landmarks)	Estimate finger states and map the pose.	Rock, Paper, Scissors, or Unknown.
computer_move()	Choose a computer action.	One of the three game moves.
decide_winner(player, computer)	Apply the game rules.	Player win, computer win, or draw.
update_score(result)	Update the relevant counter.	New score values.

7.3 Implementation Notes

The hand detector should be initialised once rather than recreated for every frame. The camera loop should release resources when the user exits. Gesture classification should be tested independently with recorded landmark examples or carefully staged hand poses. The game logic should use explicit comparisons and should reject unsupported input values. If the selected hand-landmark library changes, only the detection adapter should need substantial modification.

7.4 Installation and Execution (Typical)

Create a Python environment, install the selected dependencies, and run the main application script from the project folder. A typical setup uses OpenCV and MediaPipe packages installed through pip. Package versions may vary by operating system and Python version, so the final requirements file should record the versions that work on the lab computer. The webcam must not be in exclusive use by another application.

8. Testing, Results and Analysis

8.1 Test Plan

Test case	Input / condition	Expected behaviour
TC-1	No hand in camera view	Display a prompt; do not start a round.
TC-2	Closed fist held clearly	Classify as Rock.
TC-3	Open palm held clearly	Classify as Paper.
TC-4	Index and middle fingers extended	Classify as Scissors.
TC-5	Unclear or partially hidden hand	Return Unknown or wait for a clearer pose.
TC-6	Same player and computer move	Declare a draw and increment draw count.
TC-7	Player move beats computer move	Declare player win and update player score.
TC-8	Computer move beats player move	Declare computer win and update computer score.
TC-9	Quit application	Release camera and close the application safely.

8.2 Game Logic Verification

The game engine has nine possible player/computer combinations. These combinations can be tested using a fixed test table: three draws, three player wins, and three computer wins. The outcome should be deterministic for a given pair of moves. This test does not depend on gesture recognition and can identify errors in the rule implementation separately.

8.3 Recognition Evaluation

Gesture recognition should be evaluated by presenting each of the three gestures several times under different conditions, such as slightly different distances, hand orientations, and lighting levels. Record the number of correct classifications, incorrect classifications, and Unknown results. Recognition accuracy can be calculated as correct classifications divided by total valid gesture attempts, multiplied by 100. The report does not claim a measured accuracy value because the result depends on the actual camera, environment, and completed test run.

8.4 Result Interpretation

A successful demonstration should show the webcam preview, recognise the three supported poses, generate a computer move, and display a correct round result. The test plan also checks that the application does not score a round when the hand is absent or the gesture is unclear. Any measured accuracy, response time, or test counts should be filled in after running the finished application on the submission computer.

9. Verification, Limitations and Future Scope

9.1 Verification Checklist

- Confirm that the webcam opens and frames are read successfully.
- Check that the hand detector returns landmarks when the hand is visible.
- Test Rock, Paper, and Scissors independently.
- Confirm that unclear poses are not forced into a valid move.
- Test all nine combinations in the game-rule table.
- Verify that score counters update once per completed round.
- Confirm that the camera is released when the application closes.

9.2 Limitations

- Recognition quality depends on lighting, camera resolution, hand visibility, and background conditions.
- A hand may be misclassified when fingers overlap or are outside the camera frame.
- The basic rule-based classifier supports only three predefined poses.
- Random computer moves do not learn from the player's behaviour.
- Performance may vary across computers and library versions.
- The application requires a working webcam and local Python setup.

9.3 Future Scope

Future versions could improve recognition by using more robust geometric features, confidence thresholds, and temporal smoothing across frames. Additional features may include a countdown before each round, best-of-five mode, sound effects, difficulty settings, a graphical score history, and support for multiple camera resolutions. The project could also be extended to other gesture-controlled applications, while keeping the current game as a simple test environment.

9.4 Conclusion

Rock Paper Scissors Using Gesture Control demonstrates a practical application of computer vision and gesture-based human-computer interaction. It converts a live camera image into hand landmarks, interprets those landmarks as a supported game gesture, and uses the recognised move in a familiar decision-making task. The project is suitable for an AI lab demonstration because its stages are visible, its rules are easy to verify, and its modules can be tested independently. Its main learning value lies in connecting visual perception with a real-time interactive outcome.

References

[1] OpenCV. “OpenCV-Python Tutorials.” OpenCV Documentation. https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html

[2] Google AI Edge / MediaPipe. “Hand Landmarker Guide.” https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker

[3] Python Software Foundation. “Python 3 Documentation.” https://docs.python.org/3/

[4] Bradski, G. “The OpenCV Library.” Dr. Dobb’s Journal of Software Tools, 2000.

[5] Russell, S. J., and Norvig, P. Artificial Intelligence: A Modern Approach, 4th edition. Pearson, 2021.

[6] Szeliski, R. Computer Vision: Algorithms and Applications, 2nd edition. Springer, 2022. https://szeliski.org/Book/

Appendix A. Demonstration Screens to Add

The following screenshots may be inserted after the application is implemented and tested:

- Figure A.1 – Main application window with live webcam preview.
- Figure A.2 – Rock gesture detected by the application.
- Figure A.3 – Paper gesture detected by the application.
- Figure A.4 – Scissors gesture detected by the application.
- Figure A.5 – Example round showing the player's move, computer's move, and result.

Appendix B. Local Run Procedure

1. Install a supported version of Python.
2. Create and activate a virtual environment.
3. Install OpenCV and the selected hand-landmark package using pip.
4. Connect or enable the webcam and close other applications using it.
5. Run the main Python file from the project directory.
6. Show a clear Rock, Paper, or Scissors gesture and follow the on-screen instructions.
7. Close the application using its quit control so the camera is released.

Appendix C. Submission Note

This report describes the intended system design, algorithms, requirements, and verification plan. Any implementation-specific screenshots, measured recognition accuracy, execution time, and final package versions should be added after the application has been run and verified. No unperformed experiment is represented as a completed result.

Appendix A. Representative Output Images

Important: The following screens are illustrative mockups created for the report to show the intended application layout and sample game outcomes. They are not screenshots captured from a completed, running webcam application. Replace them with actual screenshots after implementation and testing.

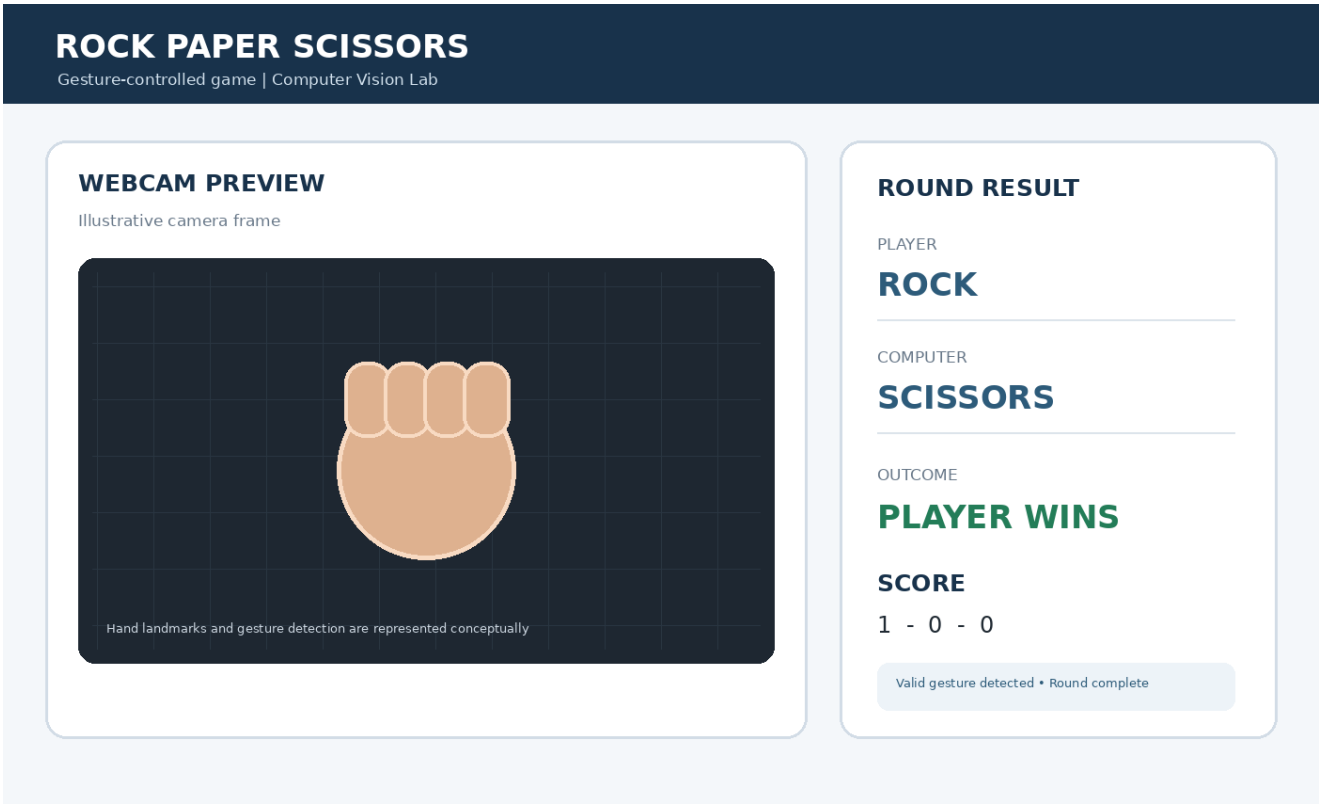


Figure A.1. Illustrative output – Rock defeats Scissors.

Appendix A. Representative Output Images (continued)

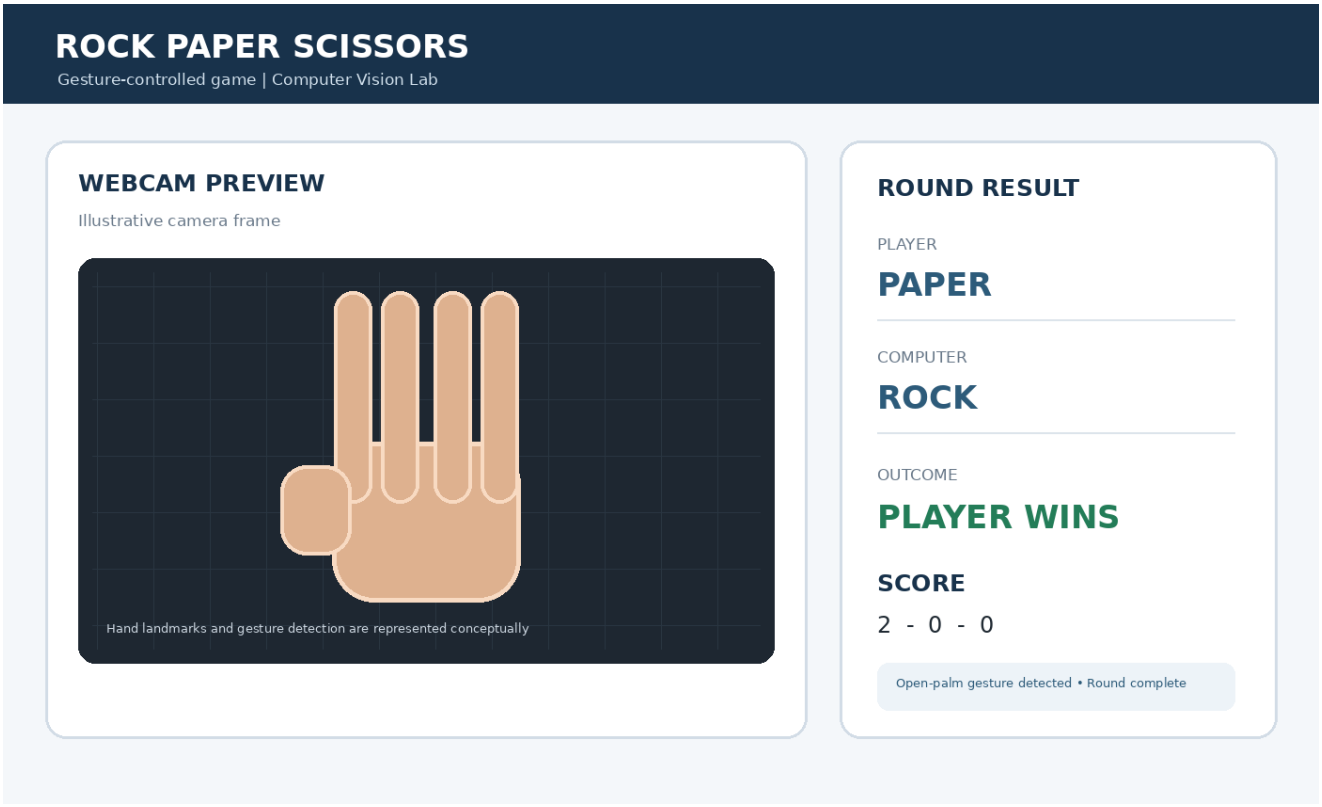


Figure A.2. Illustrative output – Paper defeats Rock.

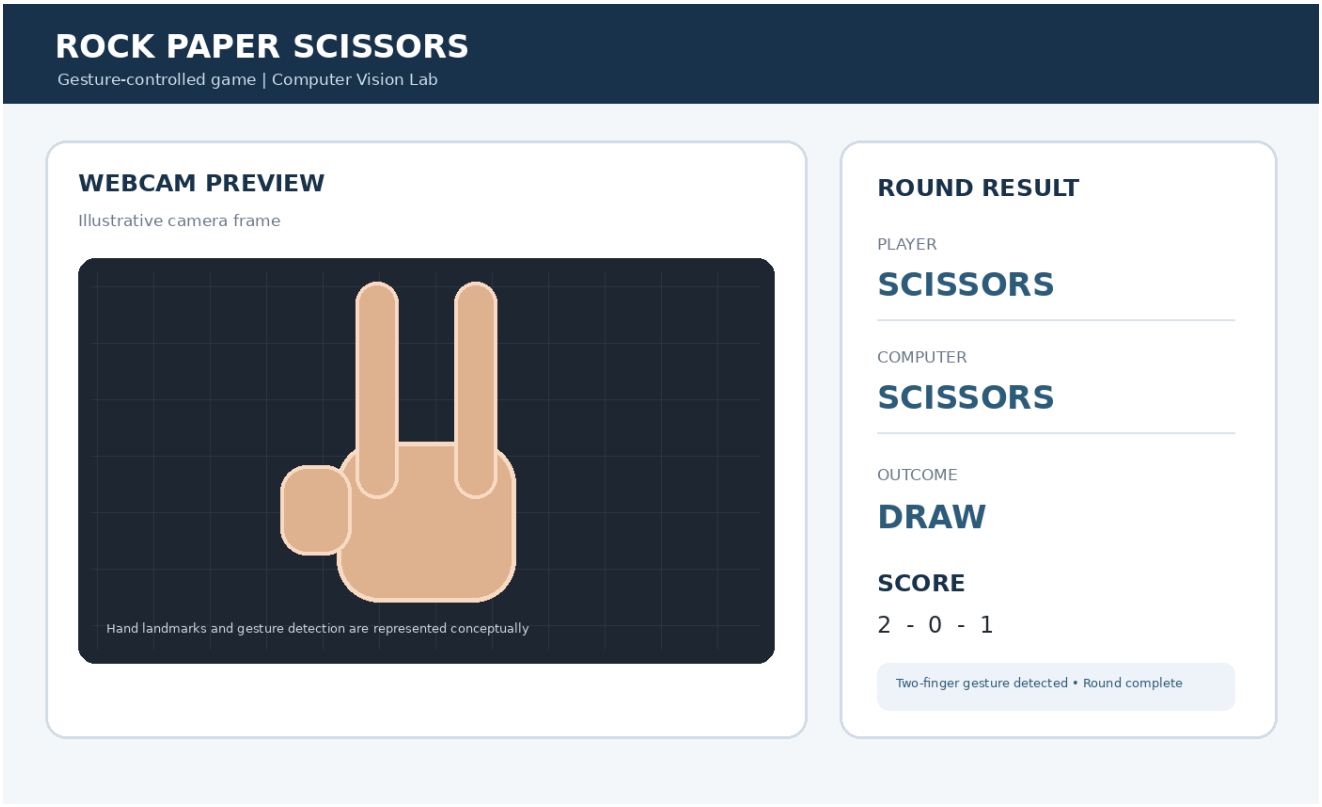


Figure A.3. Illustrative output – matching Scissors gestures result in a draw.